

Project Initialization and Planning Phase

Date	15 March 2024 - 28 June 2026
Team ID	PNT2022TMID124356_6a3a555ae5895b2c9759029d
Project Title	SmartLender - Applicant Credibility EduGenie - Google Gemini Powered Learning Assistant
Maximum Marks	5 Marks

Project Proposal (Proposed Solution) report

The proposal report aims to transform loan approval using machine learning, boosting efficiency and accuracy. It tackles system inefficiencies, promising better operations, reduced risks, and happier customers. Key features include a machine learning-based credit model and real-time decision-making.

Project Overview	
Objective	Give students practice quizzes, summarize notes, and build a personalized learning roadmap - all backed by Google Gemini with an offline/local model fallback. The primary objective is to revolutionize the loan approval process by implementing a single AI-powered assistant that answers questions, explains concepts at the right depth, generates practice quizzes, summarizes notes, and builds a personalized learning roadmap - all backed by Google Gemini with an offline/local model fallback.
Scope	The project comprehensively assesses and enhances the loan approval process, incorporating machine learning for a more robust and efficient system. It will back a single, adaptive tool that can explain a concept at their level, quiz them on it, summarize their notes, and tell them what they need to know.
Problem Statement	
Description	plan their learning path with less guesswork. plan their learning path with less guesswork. Addressing inaccuracies and inefficiencies in the current loan approval system adversely affects operational efficiency.
Impact	Use Google Gemini (with dynamic best-model resolution and retry/backoff) as the primary generative engine, prompted with fallbacks so the app is always usable. N response schemas per feature, and a local HuggingFace LaMini-Flan-T5 model plus a built-in mock/demo mode as N response schemas per feature, and a local HuggingFace LaMini-Flan-T5 model plus a built-in mock/demo mode as Solving these issues will result in improved operational efficiency, fallbacks so the app is always usable.
Proposed Solution	
Approach	Employing machine learning techniques to analyze and predict creditworthiness, creating a dynamic and adaptable loan approval model. CPU / GPU for running the local T5 model (CUDA auto-detected, uses GPU if available).
Key Features	- Implementation of a machine learning-based credit assessment model. Disk space for model weight cache, Docker image, and JSON database files (2-3 GB).

FastAPI, Uvicorn, Jinja2 template rendering engines.

google-generativeai, transformers, torch, pydantic, python-dotenv, httpx, python-multipart, pytest.

	-Real-time decision-making for quicker loan approvals. -Continuous learning to adapt to evolving financial landscapes.
--	---

Resource Requirements

Resource Type	Description	Specification/Allocation
Hardware		
Computing Resources	CPU/GPU specifications, number of cores	T4 GPU
Memory	RAM specifications	8 GB
Storage	Disk space for data, models, logs	1 TB SSD
Software		
Frameworks	Python frameworks AI-powered assistant that answers questions, explain Python	Flask
Libraries	notes, and builds a personalized learning roadmap - al Additional libraries back. Additional libraries	backed by scikit-learn, pandas, numpy,
Development Environment	IDE	Jupyter Notebook, pycharm
Data	Students lack a single, adaptive tool that can explain a concept at their level, quiz them on it, summarize their notes, and tell them what to study next - they currently stitch this together from multiple sources. Source, size, format	Local structured JSON flat-file storage logs Kaggle dataset, 614, csv dataset, 690, csv self-testing

Use Google Gemini (with dynamic best-model resolution and retry/backoff) as the primary generative engine, prompted with strict JSON response schemas per feature, and a local HuggingFace LaMini-Flan-T5 model plus a built-in mock/demo mode as fallbacks so the app is always usable.

CPU / GPU for running the local T5 model (CUDA auto-detected, uses GPU if available).

RAM to hold the local LaMini-Flan-T5 model in memory when loaded (2-4 GB recommended).

Disk space for model weight cache, Docker image, and JSON database files (2-3 GB).

FastAPI, Uvicorn, Jinja2 template rendering engines.

google-generativeai, transformers, torch, pydantic, python-dotenv, httpx, python-multipart, pytest.